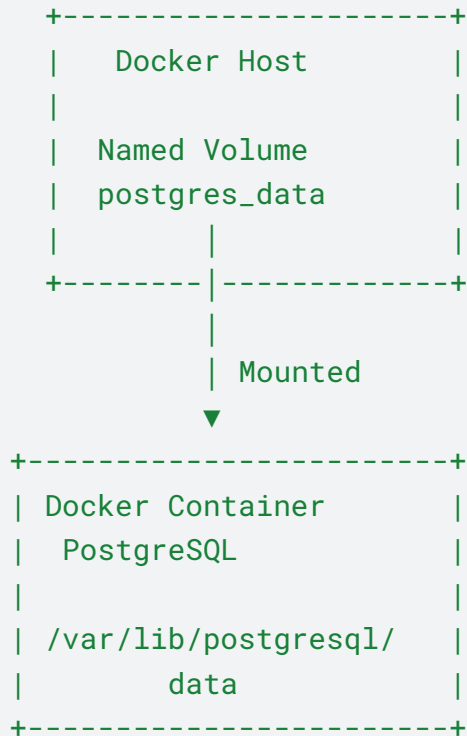


Day 32 – Docker Volumes & Networking

None

NAMED VOLUME
(Managed by Docker)



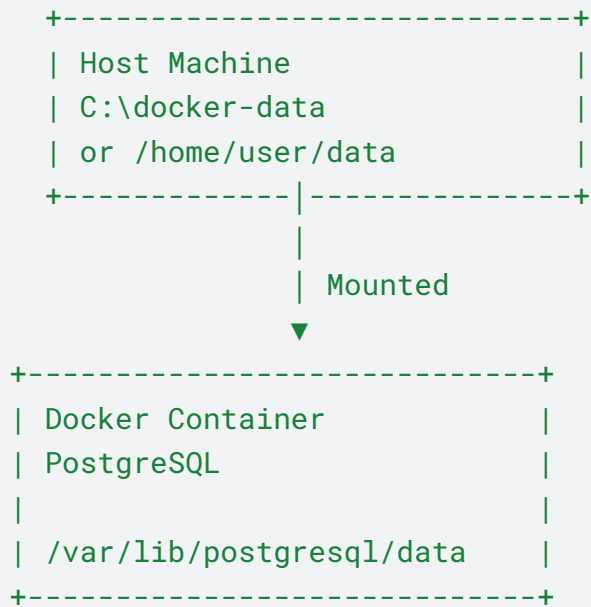
Command:

```
docker run -v postgres_data:/var/lib/postgresql/data postgres
```

None

BIND MOUNT

(Uses a folder from your computer)



Command:

```
docker run -v C:\docker-data:/var/lib/postgresql/data postgres
```

Linux:

```
docker run -v /home/user/data:/var/lib/postgresql/data postgres
```

Task 1: The Problem (Data Loss Without Volume)

Step 1: Run PostgreSQL Container

None

```
docker run -d \  
--name postgres-db \  
-e POSTGRES_PASSWORD=admin123 \  
-p 5432:5432 \  
postgres
```

Check container:

None

```
docker ps
```

Step 2: Enter PostgreSQL

None

```
docker exec -it postgres-db psql -U postgres
```

Create database and table:

None

```
CREATE DATABASE testdb;  
\c testdb
```

```
CREATE TABLE employees(  
id SERIAL PRIMARY KEY,  
name VARCHAR(50)  
);
```

```
INSERT INTO employees(name)
```

```
VALUES ('Parthavi');
```

```
SELECT * FROM employees;
```

Exit:

None

```
\q
```

```
ubuntu@ip-172-31-81-141:~/volumes$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
72fe36d92b6f   postgres "docker-entrypoint.s..." 49 seconds ago Up 43 seconds 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp postgres-db
ubuntu@ip-172-31-81-141:~/volumes$ docker exec -it postgres-db psql -U postgres
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

postgres=# CREATE DATABASE testdb;
\c testdb

CREATE TABLE employees(
id SERIAL PRIMARY KEY,
name VARCHAR(50)
);

INSERT INTO employees(name)
VALUES ('Parthavi');

SELECT * FROM employees;
CREATE DATABASE
invalid integer value "employees(" for connection option "port"
Previous connection kept
postgres=#
```

Step 3: Remove Container

None

```
docker stop postgres-db
```

```
docker rm postgres-db
```

```
ubuntu@ip-172-31-81-141:~/volumes$ docker stop postgres-db
postgres-db
ubuntu@ip-172-31-81-141:~/volumes$ docker rm postgres-db
postgres-db
ubuntu@ip-172-31-81-141:~/volumes$
```

Step 4: Run New Container

None

```
docker run -d \
```

```
--name postgres-db-new \
```

```
-e POSTGRES_PASSWORD=admin123 \  
-p 5432:5432 \  
postgres
```

Connect again:

```
None  
docker exec -it postgres-db-new psql -U postgres
```

Check database:

```
None  
\1
```

Observation

Database and table are gone.

Why?

Containers are **ephemeral**.

When the container was removed, its writable layer was also removed.

Task 2: Named Volumes

Step 1: Create Volume

```
None  
docker volume create postgres-data
```

```
ubuntu@ip-172-31-88-123:~/volume-networking$ docker volume create postgres-data  
postgres-data  
ubuntu@ip-172-31-88-123:~/volume-networking$
```

Verify:

None

```
docker volume ls
```

```
ubuntu@ip-172-31-88-123:~/volume-networking$ docker volume ls
DRIVER      VOLUME NAME
local       2b5c80ff9511c1dc463115ba03c3101f33587a43bc3715e0d2c7d7caa6609314
local       95ed89a814a3a417927e857a78646ec02c0882e3f69e1dd1e0a124c13fb6d9cf
local       b74e0266cc27e5e61a0bf45d58443c0174c9310ffd2d55d9aa756a35bee39d52
local       postgres-data
ubuntu@ip-172-31-88-123:~/volume-networking$
```

Step 2: Run PostgreSQL With Volume

None

```
docker run -d \
  --name my_postgres \
  -e POSTGRES_PASSWORD=test@123 \
  -v postgres-data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres
```

```
ubuntu@ip-172-31-88-123:~/volume-networking$ docker run -d \
  --name my_postgres \
  -e POSTGRES_PASSWORD=test@123 \
  -v postgres-data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres
1e539c0a16b385a4db4f7be4100b7c92f66d692a4d7e3064eb44403974a5838a
ubuntu@ip-172-31-88-123:~/volume-networking$
```

Step 3: Create Data

None

```
docker exec -it my_postgres psql -U postgres
```

None

```
CREATE DATABASE companydb;  
\c companydb
```

```
CREATE TABLE staff(  
id SERIAL PRIMARY KEY,  
name VARCHAR(50)  
);
```

```
INSERT INTO staff(name)  
VALUES ('Parthavi');
```

```
SELECT * FROM staff;
```

Exit:

None

```
\q
```

Step 4: Remove Container

None

```
docker stop postgres-vol  
docker rm postgres-vol
```

Step 5: Create New Container Using Same Volume

None

```
docker run -d \  
--name postgres-vol-new \  
-e POSTGRES_PASSWORD=admin123 \  
-v postgres-data:/var/lib/postgresql/data \  
-p 5432:5432 \  
postgres
```

Step 6: Verify Data

None

```
docker exec -it postgres-vol-new psql -U postgres
```

None

```
\1
```

Connect:

None

```
\c companydb  
SELECT * FROM staff;
```

Observation

Data still exists.

Why?

Volume exists independently from the container.

Verify Volume

List volumes:

None

```
docker volume ls
```

Inspect volume:

None

```
docker volume inspect postgres-data
```

Task 3: Bind Mounts

Step 1: Create Local Directory

Linux:

```
None
mkdir nginx-site
cd nginx-site
```

Create HTML file:

```
None
nano index.html
```

Add:

```
None
<h1>Hello from Host Machine</h1>
```

Save.

Step 2: Run Nginx Container

```
None
docker run -d \
--name nginx-bind \
-p 8080:80 \
-v $(pwd):/usr/share/nginx/html \
nginx
```

Step 3: Open Browser

Visit:

```
None  
http://<EC2-PUBLIC-IP>:8080
```

You should see:

```
None  
Hello from Host Machine
```

Step 4: Modify File

```
None  
nano index.html
```

Change:

```
None  
<h1>Docker Bind Mount Works!</h1>
```

Refresh browser.

Observation

Changes appear instantly.

No container restart required.

Named Volume vs Bind Mount

Named Volume	Bind Mount
Managed by Docker	Managed by Host OS
Stored in Docker directory	Stored anywhere on host
Best for databases	Best for source code
Portable	Host dependent
More secure	Direct host access

18-06-26

1. mkdir volumes
2. cd volumes
3. docker search postgres

```
ubuntu@ip-172-31-81-141:~/volumes$ docker search postgres
NAME                DESCRIPTION                                     STARS     OFFICIAL
postgres            The PostgreSQL object-relational database sy... 14935     [OK]
cimg/postgres       The PostgreSQL object-relational database sy... 9
circleci/postgres   The PostgreSQL object-relational database sy... 35
```

4. docker images

```
ubuntu@ip-172-31-81-141:~/volumes$ docker images
IMAGE                ID                DISK USAGE   CONTENT SIZE   EXTRA
devboard:latest      c1c4554d2bb5      538MB        103MB          U
flaskapp:latest       1687f15e61fe      1.64GB       421MB          U
my-first-image:latest f4b69f4e237d      254MB        77.1MB         U
my-nginx-site:latest c041aeaa307b      92.7MB       26MB           U
nginx:alpine         8b1e78743a03      93.6MB       27MB
node:20-alpine       fb4cd12c85ee      194MB        48.8MB
postgres:latest       29ee7bb30d80      650MB        168MB
python:3.14           6f473f84b09f      1.63GB       432MB
ubuntu:latest         f3d28607ddd7      160MB        45.3MB
```

5. Run the following command in your terminal to initialize and run the database:

```
docker run --name my-postgres-container -e POSTGRES_PASSWORD=Test@123 -p 5432:5432 -d postgres
```

```
ubuntu@ip-172-31-81-141:~/volumes$ docker run --name my-postgres-container -e POSTGRES_PASSWORD=Test@123 -p 5432:5432 -d postgres
397ad4df8179d7d358ce9c19183c5652bb33ba3e30c609cedad1102193356d03
ubuntu@ip-172-31-81-141:~/volumes$
```

6. Check the status of your newly created container by listing your active
docker ps

```

ubuntu@ip-172-31-81-141:~/volumes$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
397ad4df8179   postgres "docker-entrypoint.s..." 4 minutes ago    Up 4 minutes    0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp    my-postgres-container
ubuntu@ip-172-31-81-141:~/volumes$

```

- Log into the PostgreSQL terminal:

`docker exec -it my-postgres-container psql -U postgres`

```

ubuntu@ip-172-31-81-141:~/volumes$ docker exec -it my-postgres-container psql -U postgres
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

postgres=# CREATE DATABASE "parthavi-db";
CREATE DATABASE
postgres=# show database
postgres=# show database:
postgres=# \l
                                List of databases
  Name      | Owner   | Encoding | Locale Provider | Collate | Ctype   | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
parthavi-db | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          |
postgres    | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          |
template0   | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | =c/postgres +
            |         |          |                  |            |            |         |          | postgres=CTc/postgres
template1   | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | =c/postgres +
            |         |          |                  |            |            |         |          |

```

- Create db -parthavi-db

`\l-----show database`

`\q-----quit database`

- Mkdir local-data

`Cd local-data`

`pwd-----/home/ubuntu/local-data`

```

ubuntu@ip-172-31-81-141:~$ mkdir local-data
ubuntu@ip-172-31-81-141:~$ cd local-data
ubuntu@ip-172-31-81-141:~/local-data$ pwd
/home/ubuntu/local-data

```

- cd volumes(which contains docker image postgres)

`docker ps`

`docker stop 397ad4df8179 && docker rm 397ad4df8179`

`docker images`

```

ubuntu@ip-172-31-81-141:~$ cd volumes
ubuntu@ip-172-31-81-141:~/volumes$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
397ad4df8179   postgres "docker-entrypoint.s..." 14 minutes ago    Up 14 minutes    0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp    my-postgres-container
ubuntu@ip-172-31-81-141:~/volumes$ docker stop 397ad4df8179 && docker rm 397ad4df8179
397ad4df8179
397ad4df8179
ubuntu@ip-172-31-81-141:~/volumes$ docker images
IMAGE                                ID                                DISK USAGE    CONTENT SIZE    EXTRA
devboard:latest                     c1c4554d2bb5                     538MB         103MB           U
flaskapp:latest                      1687f15e61fe                     1.64GB        421MB           U
my-first-image:latest                f4b59f4e237d                     254MB         77.1MB          U
my-engine-site:latest                c041aea307b                      92.7MB        26MB            U
nginx:alpine                        8b1e78743a03                     93.6MB        27MB            U
node:20-alpine                      fb4cd12c85ee                     194MB         48.8MB          U
postgres:latest                     29ee7bb30d80                     650MB         168MB           U
python:3.14                         6f473f84b09f                     1.63GB        432MB           U
ubuntu:latest                       f3d28607ddd7                     160MB         45.3MB          U

```

- `docker inspect 29ee7bb30d80`

`"PGDATA=/var/lib/postgresql/18/docker"`

```

{
  "Env": [
    "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/lib/postgresql/18/bin",
    "GOSU_VERSION=1.19",
    "LANG=en_US.utf8",
    "PG_MAJOR=18",
    "PG_VERSION=18.4-1.pgdg13+1",
    "PGDATA=/var/lib/postgresql/18/docker"
  ],
  "Entrypoint": [
    "docker-entrypoint.sh"
  ]
}

```

12. `docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=Test@123 -v /home/ubuntu/local-data:/var/lib/postgresql/18/docker --name my-postgres-container postgres`

```
ubuntu@ip-172-31-81-141:~/volumes$ docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=Test@123 -v /home/ubuntu/local-data:/var/lib/postgresql/18/docker --name my-postgres-container postgres
0e3d06a562ced2d01ae0c3535fec6d3715b43adcf9164d315266a62ffa7616
```

13. `docker exec -it my-postgres-container psql -U postgres`
Create db- parthavi-db

```
ubuntu@ip-172-31-81-141:~/volumes$ docker exec -it my-postgres-container psql -U postgres
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

postgres=# CREATE DATABASE "parthavi-db";
CREATE DATABASE
postgres=# \l

               List of databases
  Name          | Owner   | Encoding | Locale Provider | Collate  | Ctype    | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
parthavi-db     | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |           |
postgres       | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |           |
template0      | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |           | =c/postgres      +
               |         |         |                 |             |             |         |           | postgres=CTc/postgres
template1      | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |           | =c/postgres      +
               |         |         |                 |             |             |         |           | postgres=CTc/postgres
(4 rows)

postgres=# \q
```

14. `docker ps`

```
ubuntu@ip-172-31-81-141:~/volumes$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
0e3d06a562ce   postgres  "docker-entrypoint.s..." About a minute ago Up About a minute 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp my-postgres-container
```

15. `docker stop 0e3d06a562ce && docker rm 0e3d06a562ce`

```
ubuntu@ip-172-31-81-141:~/volumes$ docker stop 0e3d06a562ce && docker rm 0e3d06a562ce
0e3d06a562ce
0e3d06a562ce
```

16. `docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=Test@123 -v /home/ubuntu/local-data:/var/lib/postgresql/18/docker --name my-postgres-container postgres`

```
ubuntu@ip-172-31-81-141:~/volumes$ docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=Test@123 -v /home/ubuntu/local-data:/var/lib/postgresql/18/docker --name my-postgres-container postgres
b5a155a864e7aea1945c7f0c8171cef9849e097c5ca3d19ee819d48f50dc8fda
```

17. `docker exec -it my-postgres-container psql -U postgres`

```
ubuntu@ip-172-31-81-141:~/volumes$ docker exec -it my-postgres-container psql -U postgres
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

postgres=# \l

               List of databases
  Name          | Owner   | Encoding | Locale Provider | Collate  | Ctype    | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
parthavi-db     | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |           |
postgres       | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |           |
template0      | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |           | =c/postgres      +
               |         |         |                 |             |             |         |           | postgres=CTc/postgres
template1      | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |           | =c/postgres      +
               |         |         |                 |             |             |         |           | postgres=CTc/postgres
(4 rows)

postgres=# ^C
postgres=#
```

18. You will data again-

```
ubuntu@ip-172-31-81-141:~$ mkdir volumes
```

```
ubuntu@ip-172-31-81-141:~$ cd volumes
```

```
ubuntu@ip-172-31-81-141:~/volumes$ docker search postgres
```

NAME	DESCRIPTION	STARS	OFFICIAL
postgres	The PostgreSQL object-relational database sy...	14935	[OK]
cimg/postgres		9	

circleci/postgres	The PostgreSQL object-relational database sy...	35
kasmweb/postgres	Postgres image maintained by Kasm Technologi...	6
elestio/postgres	Postgres, verified and packaged by Elestio	2
ubuntu/postgres	PostgreSQL is an open source object-relation...	45
cleanstart/postgres	Secure by Design, Built for Speed, Hardened ...	0
artifacthub/postgres		0
geokrety/postgres	Postgres with postgis + quantile and amqp ex...	0
corpusops/postgres	https://github.com/corpusops/docker-images/	0
rootpublic/postgres		0
dockette/postgres	My PostgreSQL image with tuning and preinst...	1
vulhub/postgres		1
wayofdev/postgres		0
pgrouting/postgres	Postgres Docker images with PostGIS and dep...	0
uselagoon/postgres		0
openeuler/postgres		0
clarinpl/postgres		0
supabase/postgres	Unmodified Postgres with some useful plugins...	77
mosipdev/postgres		0
brimstone/postgres	postgres image with traefik-cert support	0
trainlineeurope/postgres	Extended version of official Postgres https://...	0
blacklabelops/postgres	Postgres Image for Atlassian Applications	4
fredboat/postgres	PostgreSQL 10.0 used in FredBoat's docker-co...	1
sushantdhiman/postgres	Postgres Image for Sequelize	2

ubuntu@ip-172-31-81-141:~/volumes\$ docker pull postgres

Using default tag: latest

latest: Pulling from library/postgres

Digest:

sha256:29ee7bb30d804447dc9a91fd0d74322ae1dc3a4072cc6346f70a5ed6e783b565

Status: Image is up to date for postgres:latest

docker.io/library/postgres:latest

ubuntu@ip-172-31-81-141:~/volumes\$ docker images

i Info → U In Use

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
devboard:latest	c1c4554d2bb5	538MB	103MB	U
flaskapp:latest	1687f15e61fe	1.64GB	421MB	U
my-first-image:latest	f4b69f4e237d	254MB	77.1MB	U
my-nginx-site:latest	c041aeea307b	92.7MB	26MB	U
nginx:alpine	8b1e78743a03	93.6MB	27MB	
node:20-alpine	fb4cd12c85ee	194MB	48.8MB	
postgres:latest	29ee7bb30d80	650MB	168MB	
python:3.14	6f473f84b09f	1.63GB	432MB	
ubuntu:latest	f3d28607ddd7	160MB	45.3MB	

```

ubuntu@ip-172-31-81-141:~/volumes$ docker run --name my-postgres-container -e
POSTGRES_PASSWORD=Test@123 -p 5432:5432 -d postgres^C
ubuntu@ip-172-31-81-141:~/volumes$ docker run --name my-postgres-container -e
POSTGRES_PASSWORD=Test@123 -p 5432:5432 -d postgres
397ad4df8179d7d358ce9c19183c5652bb33ba3e30c609cedad1102193356d03
ubuntu@ip-172-31-81-141:~/volumes$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS         NAMES
397ad4df8179   postgres "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes
0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   my-postgres-container
ubuntu@ip-172-31-81-141:~/volumes$ docker exec -it my-postgres-container psql -U
postgres
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

```

```

postgres=# CREATE DATABASE "parthavi-db";
CREATE DATABASE
postgres=# show database
postgres=# show database:
postgres=# \l

```

List of databases

Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
parthavi-db	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
postgres	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
template0	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
template1	postgres	UTF8	libc	en_US.utf8	en_US.utf8			

(4 rows)

```

postgres=# /q
postgres=# \q
ubuntu@ip-172-31-81-141:~/volumes$ cd ..
ubuntu@ip-172-31-81-141:~$ ls
compose-basics flask-app-ecs indexexample my-first-image project2-devboard
volumes
ubuntu@ip-172-31-81-141:~$ mkdir local-data
ubuntu@ip-172-31-81-141:~$ cd local-data
ubuntu@ip-172-31-81-141:~/local-data$ pwd

```

```

/home/ubuntu/local-data
ubuntu@ip-172-31-81-141:~/local-data$ cd..
cd..: command not found
ubuntu@ip-172-31-81-141:~/local-data$ cd ..
ubuntu@ip-172-31-81-141:~$ cd volumes
ubuntu@ip-172-31-81-141:~/volumes$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS         NAMES
397ad4df8179   postgres "docker-entrypoint.s..." 14 minutes ago Up 14 minutes
0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp my-postgres-container
ubuntu@ip-172-31-81-141:~/volumes$ docker stop 397ad4df8179 && docker rm
397ad4df8179
397ad4df8179
397ad4df8179
ubuntu@ip-172-31-81-141:~/volumes$ docker images

```

i Info → U In Use

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
devboard:latest	c1c4554d2bb5	538MB	103MB	U
flaskapp:latest	1687f15e61fe	1.64GB	421MB	U
my-first-image:latest	f4b69f4e237d	254MB	77.1MB	U
my-nginx-site:latest	c041aeea307b	92.7MB	26MB	U
nginx:alpine	8b1e78743a03	93.6MB	27MB	
node:20-alpine	fb4cd12c85ee	194MB	48.8MB	
postgres:latest	29ee7bb30d80	650MB	168MB	
python:3.14	6f473f84b09f	1.63GB	432MB	
ubuntu:latest	f3d28607ddd7	160MB	45.3MB	

```

ubuntu@ip-172-31-81-141:~/volumes$ docker inspect 29ee7bb30d80
[
  {
    "Id":
"sha256:29ee7bb30d804447dc9a91fd0d74322ae1dc3a4072cc6346f70a5ed6e783b565"
,
    "RepoTags": [
      "postgres:latest"
    ],
    "RepoDigests": [
      "postgres@sha256:29ee7bb30d804447dc9a91fd0d74322ae1dc3a4072cc6346f70a5ed6e783b565"
    ],
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2026-06-11T00:34:40.564557847Z",
    "Config": {

```



```

    "ExposedPorts": {
      "5432/tcp": {}
    },
    "Env": [
      "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/lib/postgresql/18/bin",
      "GOSU_VERSION=1.19",
      "LANG=en_US.utf8",
      "PG_MAJOR=18",
      "PG_VERSION=18.4-1.pgdg13+1",
      "PGDATA=/var/lib/postgresql/18/docker"
    ],
    "Entrypoint": [
      "docker-entrypoint.sh"
    ],
    "Cmd": [
      "postgres"
    ],
    "Volumes": {
      "/var/lib/postgresql": {}
    },
    "StopSignal": "SIGINT"
  },
  "Architecture": "amd64",
  "Os": "linux",
  "Size": 162381384,
  "RootFS": {
    "Type": "layers",
    "Layers": [
      "sha256:f4f8b983b714f130e2cff99176baa26352db6a55d3622e10ada40f2b4720a4eb",
      "sha256:96d991145b2d9f64cee4894ad03c4fdb6623787cd2ac60e11392df4cb871a602",
      "sha256:f35aedbe8524aaca56c2f1d36f2c72ce746224f807c7ee986cdd184e240e516f",
      "sha256:3e171a4157d3a6de07bb1c550890471983a566c80bde2b7e3cf90ae60d4f30a2",
      "sha256:5184c7d7da0ffc54374ab54c88d3f2c1e71fa338848b970178680aad9c103d0d",
      "sha256:61102d9e145cb4f0c583a728a53b651fa1ddf8f4665ea9bce9d06429e0124f72",
      "sha256:d03fd15d6d43dc72908f2f04c3877e99c15369f188e33ba1d1c7d437afd21261",

```

```

"sha256:c0495d78628235d443221203463afa75d78bde15dc893c27ffa3a5eac9abdaa1",
"sha256:b92f948f89730567ac46619854ab581b64f1ef97584525754979f8d319964797",
"sha256:3a3d476d6f94dd27e8fc9afdb0798864d6b54828d6b2caf94c090512c51855c8",
"sha256:9bdeffaab6fbddca253881812fe8021d8b74ada5fb1941bc141c8a8f41de9d04",
"sha256:5820cea328510db5345943de3b31bce3709f2dbc4f30f0fc1c2f00b40a580981",
"sha256:00952b970abbae81955eac406b1dac18313ad96c8ccafcf0a41e3b6ad0238182"
  ],
  "Metadata": {
    "LastTagTime": "2026-06-18T03:29:32.070246276Z"
  },
  "Descriptor": {
    "mediaType": "application/vnd.oci.image.index.v1+json",
    "digest":
"sha256:29ee7bb30d804447dc9a91fd0d74322ae1dc3a4072cc6346f70a5ed6e783b565"
  },
  "size": 10229
}
]

```

```

ubuntu@ip-172-31-81-141:~/volumes$ docker run -d -p 5432:5432 -e
POSTGRES_PASSWORD=Test@123 -v
/home/ubuntu/local-data:/var/lib/postgresql/18/docker --name my-postgres-container
postgres
0e3d06a562ced2d01ae0c3535fec6d3715b43adcafd9164d315266a62ffa7616
ubuntu@ip-172-31-81-141:~/volumes$ docker exec -it my-postgres-container psql -U
postgres
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

```

```

postgres=# CREATE DATABASE "parthavi-db";
CREATE DATABASE
postgres=# \l

```

List of databases

Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
------	-------	----------	-----------------	---------	-------	--------	-----------	-------------------

```

-----+-----+-----+-----+-----+-----+-----+-----
-----

```

```

parthavi-db | postgres | UTF8 | libc | en_US.utf8 | en_US.utf8 | | |
postgres | postgres | UTF8 | libc | en_US.utf8 | en_US.utf8 | | |
template0 | postgres | UTF8 | libc | en_US.utf8 | en_US.utf8 | | |
=c/postgres +
| | | | | | | | postgres=CtC/postgres
template1 | postgres | UTF8 | libc | en_US.utf8 | en_US.utf8 | | |
=c/postgres +
| | | | | | | | postgres=CtC/postgres
(4 rows)

```

```

postgres=# \q
ubuntu@ip-172-31-81-141:~/volumes$ dockerps
dockerps: command not found
ubuntu@ip-172-31-81-141:~/volumes$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS         NAMES
0e3d06a562ce   postgres "docker-entrypoint.s..." About a minute ago Up About a
minute        0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp my-postgres-container
ubuntu@ip-172-31-81-141:~/volumes$ docker stop 0e3d06a562ce && docker rm
0e3d06a562ce
0e3d06a562ce
0e3d06a562ce
ubuntu@ip-172-31-81-141:~/volumes$ docker run -d -p 5432:5432 -e
POSTGRES_PASSWORD=Test@123 -v
/home/ubuntu/local-data:/var/lib/postgresql/18/docker --name my-postgres-container
postgres
b5a155a864e7aea1945c7f0c8171cef9849e097c5ca3d19ee819d48f50dc8fda
ubuntu@ip-172-31-81-141:~/volumes$ docker exec -it my-postgres-container psql -U
postgres
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

```

```
postgres=# \l
```

List of databases

Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
parthavi-db	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
postgres	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
template0	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
=c/postgres	+							
								postgres=CtC/postgres

```
template1 | postgres | UTF8 | libc | en_US.utf8 | en_US.utf8 | | |
=c/postgres +
(4 rows)

postgres=# ^C
postgres=# \q
ubuntu@ip-172-31-81-141:~/volumes$
```

Task 4: Docker Networking Basics

Step 1: List Networks

```
None
docker network ls
```

Example:

```
None
bridge
host
none
```

Step 2: Inspect Bridge Network

```
None
docker network inspect bridge
```

Step 3: Run Two Containers

```
None
docker run -dit --name ubuntu1 ubuntu
docker run -dit --name ubuntu2 ubuntu
```

Step 4: Install Ping

Inside ubuntu1:

None

```
docker exec -it ubuntu1 bash
```

None

```
apt update  
apt install iputils-ping -y
```

Exit.

Do same for ubuntu2.

Step 5: Test Name Resolution

None

```
docker exec -it ubuntu1 ping ubuntu2
```

Expected:

None

```
ping: ubuntu2: Name or service not known
```

Step 6: Test By IP

Find IP:

None

```
docker inspect ubuntu2
```

Look for:

None

```
172.17.x.x
```

Ping:

None

```
docker exec -it ubuntu1 ping 172.17.x.x
```

Expected:

None

```
PING successful
```

Observation

Default bridge network does not provide automatic DNS between containers.

Task 5: Custom Networks

Step 1: Create Network

None

```
docker network create my-app-net
```

Verify:

None

```
docker network ls
```

Step 2: Run Containers

None

```
docker run -dit \  
--name app1 \  
--network my-app-net \  
ubuntu
```

None

```
docker run -dit \  
--name app2 \  
--network my-app-net \  
ubuntu
```

Step 3: Install Ping

None

```
docker exec -it app1 bash
```

None

```
apt update  
apt install iputils-ping -y
```

Step 4: Ping By Name

None

```
ping app2
```

Expected:

None

```
PING app2 (172.xx.xx.xx)
```

Success.

Why Does Custom Networking Work?

Docker automatically provides an internal DNS service for user-created bridge networks.

Therefore containers can communicate using container names.

Default bridge network lacks automatic DNS resolution.

Task 6: Put It Together

Create Network

None

```
docker network create my-db-network
```

Run PostgreSQL With Volume

None

```
docker volume create pgdata
```

None

```
docker run -d \  
--name postgres-final \  
--network my-db-network \  
-e POSTGRES_PASSWORD=admin123 \  
-v pgdata:/var/lib/postgresql/data \  
postgres
```

Run Application Container

None

```
docker run -dit \  
--name app-final \  
--network my-db-network \  
ubuntu
```

Install PostgreSQL Client

None

```
docker exec -it app-final bash
```

None

```
apt update  
apt install postgresql-client -y
```

Connect Using Container Name

None

```
psql -h postgres-final -U postgres
```

Password:

None

```
admin123
```

If connected successfully:

None

```
postgres=#
```

Verification

Application container can reach database container using:

None

postgres-final

instead of IP.

Key Learning Summary

- ✓ Containers lose data when removed
- ✓ Named volumes persist data
- ✓ Bind mounts sync host files into containers
- ✓ Default bridge allows IP communication
- ✓ Custom bridge provides DNS-based communication
- ✓ Volumes + Networks are essential for real-world applications
- ✓ Multi-container applications rely on custom networks and persistent volumes